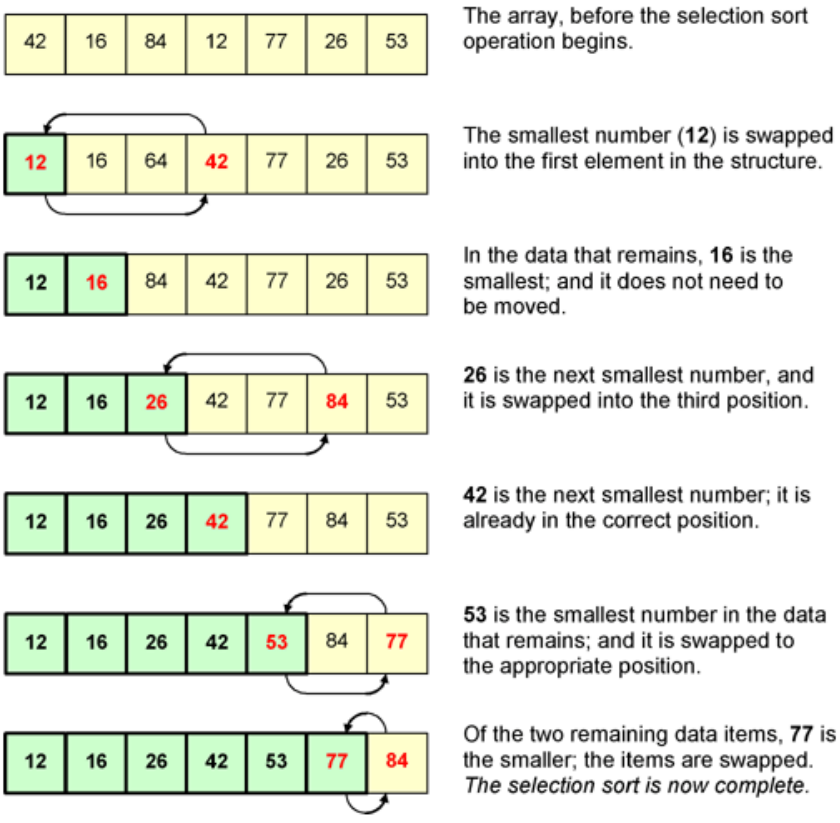
CC-4: Unit-4

1. Selection Sort:

- The selection sort algorithm sorts an array by **repeatedly finding the minimum element** (considering ascending order) from the unsorted part and putting it at the beginning.
- The algorithm maintains two sub-arrays in a given array:
    1. The sub-array which is **already sorted**.
    2. Remaining sub-array which is **unsorted**.
- In every iteration of selection sort, the minimum element (considering ascending order) from the unsorted sub-array is picked and moved to the sorted sub-array.

Example:



Algorithm:

1. Step 1 – Set MIN to location 0
2. Step 2 – Search the minimum element in the list
3. Step 3 – Swap with value at location MIN
4. Step 4 – Increment MIN to point to next element
5. Step 5 – Repeat until list is sorted

C Program:

```
#include <stdio.h>
void swap(int *xp, int *yp)
{
      int temp = *xp;
      *xp = *yp;
      *yp = temp;
}
void selectionSort(int arr[], int n)
{
      int i, j, min_idx;
      for (i = 0; i < n-1; i++)
      {
            min_idx = i;
            for (j = i+1; j < n; j++)
            if (arr[j] < arr[min_idx])
                  min_idx = j;
            swap(&arr[min_idx], &arr[i]);
      }
}
void printArray(int arr[], int size)
{
      int i;
      for (i=0; i < size; i++)
            printf("%d ", arr[i]);
      printf("\n");
}
int main()
{
      int arr[] = {64, 25, 12, 22, 11};
      int n = sizeof(arr)/sizeof(arr[0]);
      selectionSort(arr, n);
      printf("Sorted array: \n");
      printArray(arr, n);
      return 0;
}
```

Output:

```
Sorted array:
11 12 22 25 64
```

Number of comparisons:

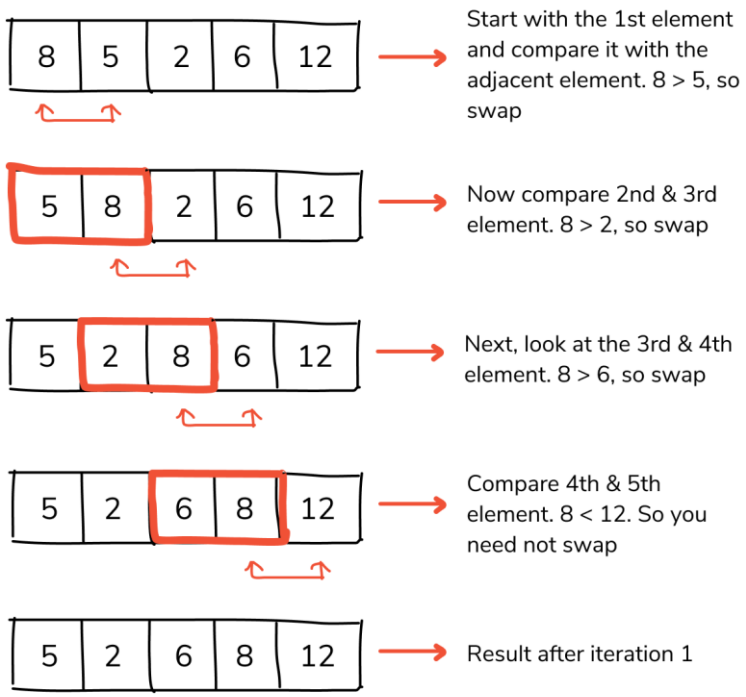
`(n-1) + (n-2) + (n-3) +.....+ 1 = n(n-1)/2` nearly equals to n^2

Complexity = $O(n^2)$

2. Bubble sort:

- Bubble Sort is a simple algorithm which is used to sort a given set of **n** elements provided in form of an array. Bubble Sort compares the entire element one by one and sort them based on their values.
- If the given array has to be sorted in ascending order, then bubble sort will start by comparing the first element of the array with the second element, if the first element is greater than the second element, it will swap both the elements, and then move on to compare the second and the third element, and so on.
- If we have total n elements, then we need to repeat this process for n-1 times.
- It is known as bubble sort, because with every complete iteration the largest element in the given array, bubbles up towards the last place or the highest index, just like a water bubble rises up to the water surface.

Example:



Algorithm:

1. Run a nested for loop to traverse the input array using two variables i and j, such that $0 \leq i < n-1$ and $0 \leq j < n-i-1$
2. If arr[j] is greater than arr[j+1] then swap these adjacent elements, else move on
3. Print the sorted array

C Program:

```
#include <stdio.h>
void swap(int *xp, int *yp)
{
      int temp = *xp;
      *xp = *yp;
      *yp = temp;
}

void bubbleSort(int arr[], int n)
{
int i, j;
for (i = 0; i < n-1; i++)

      // Last i elements are already in place
      for (j = 0; j < n-i-1; j++)
            if (arr[j] > arr[j+1])
                  swap(&arr[j], &arr[j+1]);
}

void printArray(int arr[], int size)
{
      int i;
      for (i=0; i < size; i++)
            printf("%d ", arr[i]);
      printf("\n");
}

int main()
{
      int arr[] = {64, 34, 25, 12, 22, 11, 90};
      int n = sizeof(arr)/sizeof(arr[0]);
      bubbleSort(arr, n);
      printf("Sorted array: \n");
      printArray(arr, n);
      return 0;
}
```

Output: Sorted array:

11 12 22 25 34 64 90

Complexity:

We can analyze the complexity by simply observing the number of loops. There are 2 loops so the complexity is $n*n = n^2$

Example:

i = 0	j	0	1	2	3	4	5	6	7
	0	5	3	1	9	8	2	4	7
	1	3	5	1	9	8	2	4	7
	2	3	1	5	9	8	2	4	7
	3	3	1	5	9	8	2	4	7
	4	3	1	5	8	9	2	4	7
	5	3	1	5	8	2	9	4	7
	6	3	1	5	8	2	4	9	7
i =1	0	3	1	5	8	2	4	7	9
	1	1	3	5	8	2	4	7	
	2	1	3	5	8	2	4	7	
	3	1	3	5	8	2	4	7	
	4	1	3	5	2	8	4	7	
	5	1	3	5	2	4	8	7	
i = 2	0	1	3	5	2	4	7	8	
	1	1	3	5	2	4	7		
	2	1	3	5	2	4	7		
	3	1	3	2	5	4	7		
	4	1	3	2	4	5	7		
i = 3	0	1	3	2	4	5	7		
	1	1	3	2	4	5			
	2	1	2	3	4	5			
	3	1	2	3	4	5			
i = 4	0	1	2	3	4	5			
	1	1	2	3	4				
	2	1	2	3	4				
i = 5	0	1	2	3	4				
	1	1	2	3					
i = 6	0	1	2	3					
		1	2						

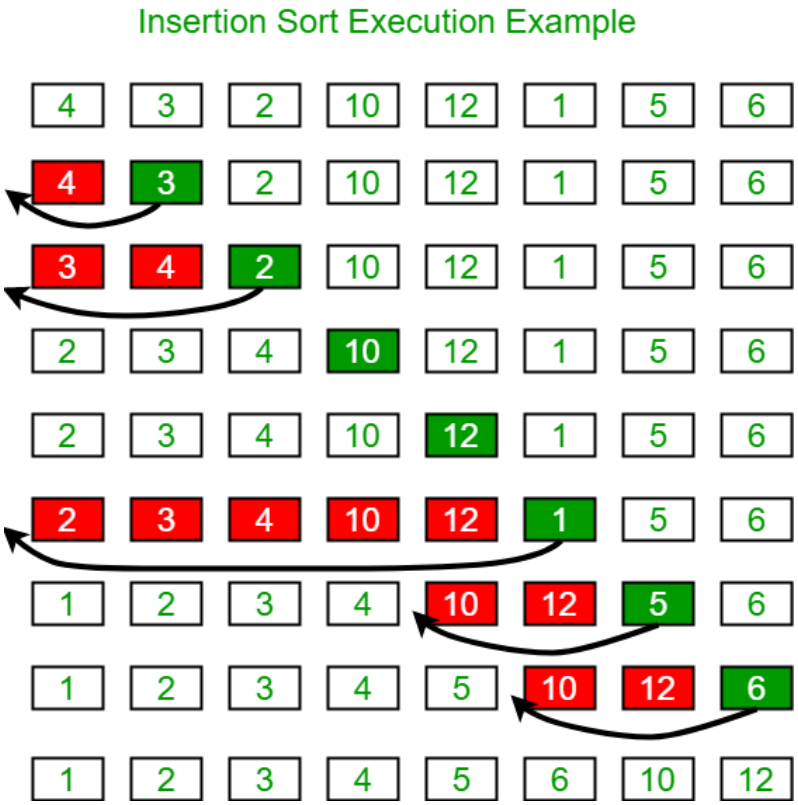
3. Insertion Sort:

- Insertion sort is a simple sorting algorithm that works similar to the way you sort playing cards in your hands.
- The array is virtually split into a sorted and an unsorted part. Values from the unsorted part are picked and placed at the correct position in the sorted part.

Algorithm

To sort an array of size N in ascending order:

1. Iterate from arr[1] to arr[N] over the array.
2. Compare the current element (key) to its predecessor.
3. If the key element is smaller than its predecessor, compare it to the elements before. Move the greater elements one position up to make space for the swapped element.



C Program:

```
#include <math.h>
#include <stdio.h>

/* Function to sort an array using insertion sort*/
void insertionSort(int arr[], int n)
{
	int i, key, j;
	for (i = 1; i < n; i++) {
		key = arr[i];
		j = i - 1;

		while (j >= 0 && arr[j] > key) {
			arr[j + 1] = arr[j];
			j = j - 1;
		}
		arr[j + 1] = key;
	}
}

// A utility function to print an array of size n
void printArray(int arr[], int n)
{
	int i;
	for (i = 0; i < n; i++)
```

```
            printf("%d ", arr[i]);
      printf("\n");
}

/* Driver program to test insertion sort */
int main()
{
      int arr[] = { 12, 11, 13, 5, 6 };
      int n = sizeof(arr) / sizeof(arr[0]);

      insertionSort(arr, n);
      printArray(arr, n);

      return 0;
}
```

Complexity:

Total comparisons: T (n) = 1+2+3+.....+ n-1
= (n-1)n / 2
= O (n^2)
Therefore, complexity is of order n^2

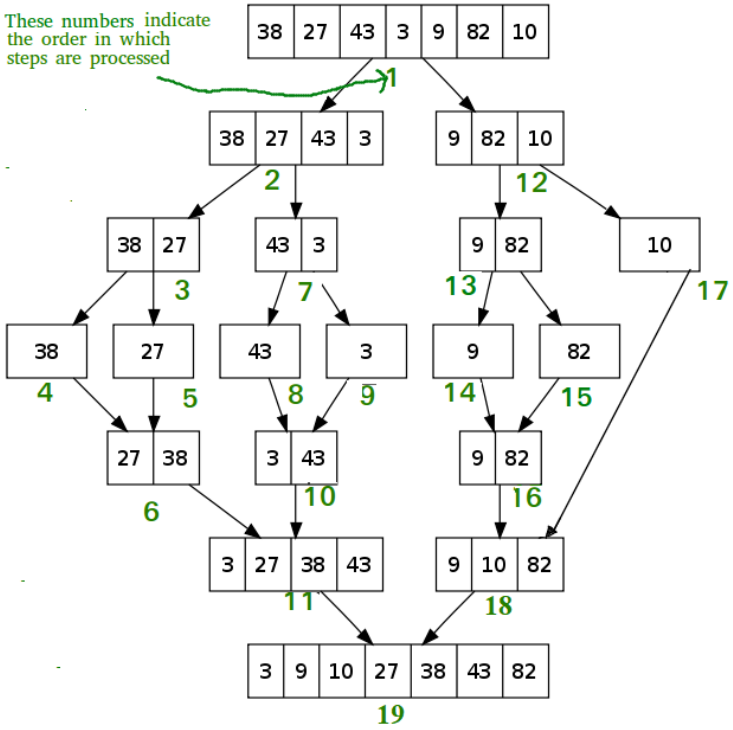
4. Merge Sort:

Merge sort follows the divide & conquer paradigm. Conceptually, it works as follows:

1. **Divide:** Divide the unsorted list into two sub-lists of about half the size.
2. **Conquer:** Sort each of the two sub-lists recursively until we have list sizes of length 1, in which case the list items are returned.
3. **Combine:** Join the two sorted Sub lists back into one sorted list.

Algorithm:

1. Step 1: declare array and left, right, mid variable
2. Step 2: perform merge function.
   if left > right
      return
   mid= (left+right)/2
   mergesort(array, left, mid)
   mergesort(array, mid+1, right)
   merge(array, left, mid, right)
3. Step 3: Stop



C Program

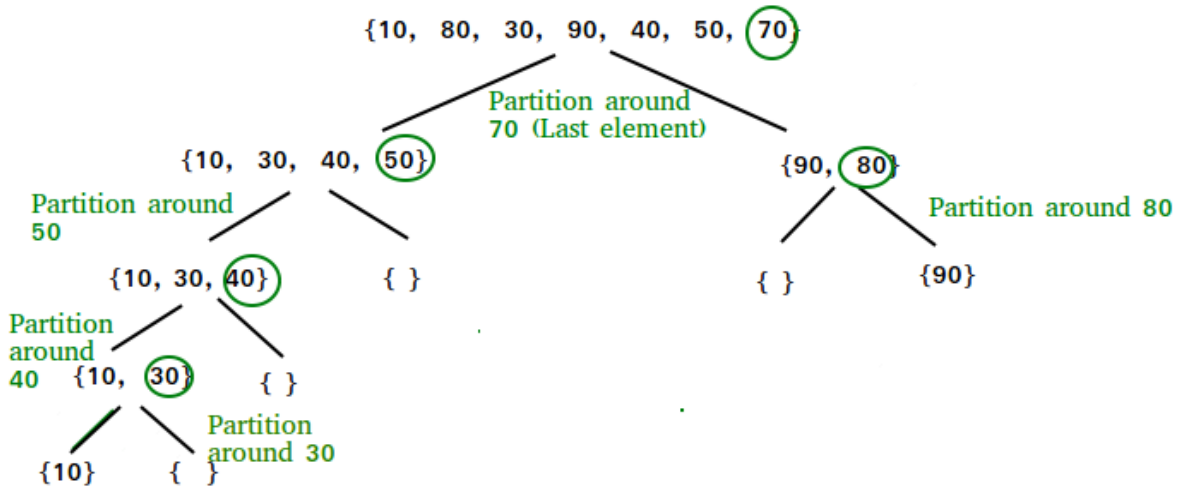
```
#include <stdio.h>
#include <stdlib.h>
void merge(int arr[], int l, int m, int r)
{
      int i, j, k;
      int n1 = m - l + 1;
      int n2 = r - m;
      int L[n1], R[n2];
      for (i = 0; i < n1; i++)
            L[i] = arr[l + i];
      for (j = 0; j < n2; j++)
            R[j] = arr[m + 1 + j];
      i = 0; // Initial index of first subarray
      j = 0; // Initial index of second subarray
      k = l; // Initial index of merged subarray
      while (i < n1 && j < n2) {
            if (L[i] <= R[j]) {
                  arr[k] = L[i];
                  i++;
            }
            else {
                  arr[k] = R[j];
```

```
                j++;
            }
            k++;
        }
        while (i < n1) {
            arr[k] = L[i];
            i++;
            k++;
        }
        while (j < n2) {
            arr[k] = R[j];
            j++;
            k++;
        }
}
void mergeSort(int arr[], int l, int r)
{
        if (l < r) {
            int m = l + (r - l) / 2;
            mergeSort(arr, l, m);
            mergeSort(arr, m + 1, r);
            merge(arr, l, m, r);
        }
}
void printArray(int A[], int size)
{
        int i;
        for (i = 0; i < size; i++)
            printf("%d ", A[i]);
        printf("\n");
}
int main()
{
        int arr[] = { 12, 11, 13, 5, 6, 7 };
        int arr_size = sizeof(arr) / sizeof(arr[0]);
        printf("Given array is \n");
        printArray(arr, arr_size);
        mergeSort(arr, 0, arr_size - 1);
        printf("\nSorted array is \n");
        printArray(arr, arr_size);
        return 0;
}
```

- The time complexity of Merge Sort is **θ(nLogn)**

5. Quick Sort:

- Like Merge Sort, **Quick Sort** is a Divide and Conquer algorithm. It picks an element as a **pivot** and partitions the given array around the picked pivot.
- There are many different versions of quick sort that pick pivot in different ways.
  - Always pick the first element as a pivot.
  - Always pick the last element as a pivot (implemented below)
  - Pick a random element as a pivot.
  - Pick median as the pivot.
- The key process in **quick sort** is a partition (). The target of partitions is, given an array and an element x of an array as the pivot.
- Put x at its correct position in a sorted array and put all smaller elements (smaller than x) before x, and put all greater elements (greater than x) after x.
- All this should be done in linear time.



Algorithm

```
/* low  –> Starting index,  high  –> Ending index */
quickSort(arr[], low, high) {
   if (low < high) {
      /* pi is partitioning index, arr[pi] is now at right place */
      pi = partition(arr, low, high);
      quickSort(arr, low, pi – 1);  // Before pi
      quickSort(arr, pi + 1, high); // After pi
   }
}
```

Algorithm for partition()

```
partition (arr[], low, high)
{
   // pivot (Element to be placed at right position)
   pivot = arr[high];
   i = (low – 1)  // Index of smaller element and indicates the
   // right position of pivot found so far
   for (j = low; j <= high- 1; j++){
      // If current element is smaller than the pivot
      if (arr[j] < pivot){
         i++;    // increment index of smaller element
         swap arr[i] and arr[j]
      }
   }
   swap arr[i + 1] and arr[high])
   return (i + 1)
}
```

C Program

```
#include <stdio.h>
void swap(int *a, int *b) {
  int t = *a;
  *a = *b;
  *b = t;
}
int partition(int array[], int low, int high) {

  int pivot = array[high];
  int i = (low - 1);
  for (int j = low; j < high; j++) {
    if (array[j] <= pivot) {
      i++;
          swap(&array[i], &array[j]);
    }
  }
  swap(&array[i + 1], &array[high]);
  return (i + 1);
}

void quickSort(int array[], int low, int high) {
  if (low < high) {
```

```
    int pi = partition(array, low, high);
    quickSort(array, low, pi - 1);
    quickSort(array, pi + 1, high);
  }
}

// function to print array elements
void printArray(int array[], int size) {
  for (int i = 0; i < size; ++i) {
    printf("%d  ", array[i]);
  }
  printf("\n");
}

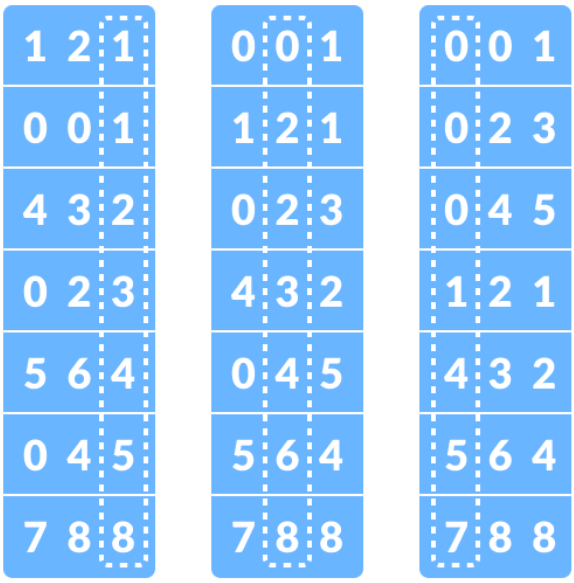
// main function
int main() {
  int data[] = {8, 7, 2, 1, 0, 9, 6};
  int n = sizeof(data) / sizeof(data[0]);
  printf("Unsorted Array\n");
  printArray(data, n);
  quickSort(data, 0, n - 1);
  printf("Sorted array in ascending order: \n");
  printArray(data, n);
}
```

Time Complexity

- **Best Case Complexity -** In Quicksort, the best-case occurs when the pivot element is the middle element or near to the middle element. The best-case time complexity of quicksort is **O(n*logn)**.
- **Average Case Complexity -** It occurs when the array elements are in jumbled order that is not properly ascending and not properly descending. The average case time complexity of quicksort is **O(n*logn)**.
- **Worst Case Complexity -** In quick sort, worst case occurs when the pivot element is either greatest or smallest element. Suppose, if the pivot element is always the last element of the array, the worst case would occur when the given array is sorted already in ascending or descending order. The worst-case time complexity of quicksort is **$O(n^2)$**.

6. Radix Sort:

- Radix sort is a sorting algorithm that sorts the elements by first grouping the individual digits of the same place value. Then, sort the elements according to their increasing or decreasing order.
- Suppose, we have an array of 8 elements. First, we will sort elements based on the value of the **unit place**. Then, we will sort elements based on the value of the **tenth place**. This process goes on until the last significant place.
- Let the initial array be [121, 432, 564, 23, 1, 45, 788]. It is sorted according to radix sort as shown in the figure below.



sorting the integers according to units, tens and hundreds place digits

Algorithm

1. radixSort(arr)
2. **max** = largest element in the given array
3. d = number of digits in the largest element (or, max)
4. Now, create d buckets of size 0 - 9
5. **for** i -> 0 to d
6. sort the array elements using counting sort (or any stable sort) according to the digits at the ith place

C Program

```
#include <stdio.h>
int getMax(int a[], int n) {
   int max = a[0];
   for(int i = 1; i<n; i++) {
      if(a[i] > max)
         max = a[i];
   }
   return max; //maximum element from the array
}
void countingSort(int a[], int n, int place) // function to
implement counting sort
{
  int output[n + 1];
  int count[10] = {0};
  // Calculate count of elements
  for (int i = 0; i < n; i++)
    count[(a[i] / place) % 10]++;
  // Calculate cumulative frequency
  for (int i = 1; i < 10; i++)
    count[i] += count[i - 1];
  // Place the elements in sorted order
  for (int i = n - 1; i >= 0; i--) {
    output[count[(a[i] / place) % 10] - 1] = a[i];
    count[(a[i] / place) % 10]--;
  }
  for (int i = 0; i < n; i++)
    a[i] = output[i];
}
// function to implement radix sort
void radixsort(int a[], int n) {
  // get maximum element from array
  int max = getMax(a, n);
  // Apply counting sort to sort elements based on place value
  for (int place = 1; max / place > 0; place *= 10)
    countingSort(a, n, place);
}
```

```
// function to print array elements
void printArray(int a[], int n) {
  for (int i = 0; i < n; ++i) {
    printf("%d  ", a[i]);
  }
  printf("\n");
}

int main() {
  int a[] = {181, 289, 390, 121, 145, 736, 514, 888, 122};
  int n = sizeof(a) / sizeof(a[0]);
  printf("Before sorting array elements are - \n");
  printArray(a,n);
  radixsort(a, n);
  printf("After applying Radix sort, the array elements are - \n");
  printArray(a, n);
}
```

Time Complexity

- **Best Case Complexity -** It occurs when there is no sorting required, i.e. the array is already sorted. The best-case time complexity of Radix sort is **Ω(n+k)**.
- **Average Case Complexity -** It occurs when the array elements are in jumbled order that is not properly ascending and not properly descending. The average case time complexity of Radix sort is **θ(nk)**.
- **Worst Case Complexity -** It occurs when the array elements are required to be sorted in reverse order. That means suppose you have to sort the array elements in ascending order, but its elements are in descending order. The worst-case time complexity of Radix sort is **O(nk)**.

7. Heap Sort:

- Heap sort is implemented using a max-heap. In a maxheap, the maximum element is at the root of the tree and is also the first element of the array i.e., A[1].

Algorithm:

- The Heap sort algorithm to arrange a list of elements in ascending order is performed using following steps.

  Step 1 - Construct a Binary Tree with given list of Elements.

  Step 2 - Transform the Binary Tree into Max Heap.

  Step 3 - Delete the root element from Max Heap using Heapify method.

  Step 4 - Put the deleted element into the Sorted list (i.e. at the last position).

  Step 5 - Decrease the Heap size by 1.

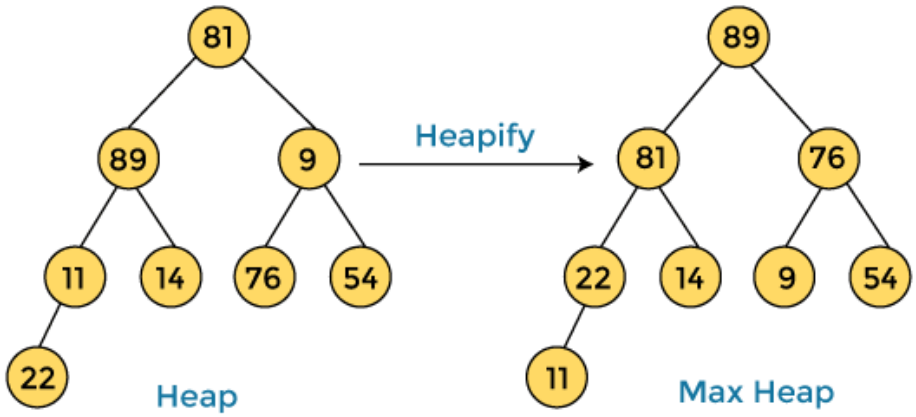
  Step 6 - Repeat the same until Min Heap becomes empty.

  Step 7 - Display the sorted list.

Example:

81	89	9	11	14	76	54	22

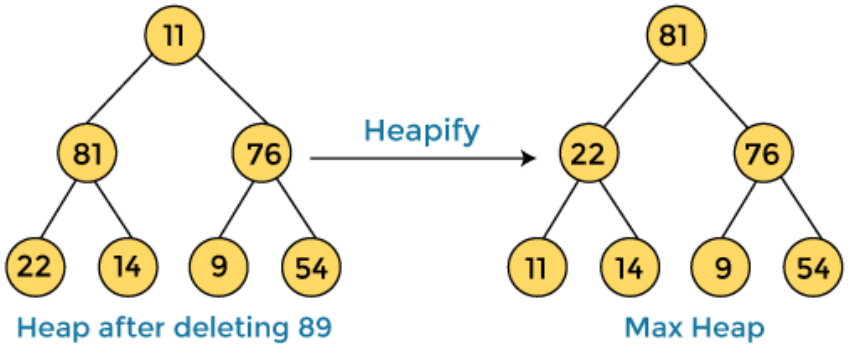
First, we have to construct a heap from the given array and convert it into max heap.



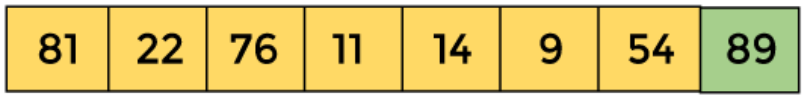
After converting the given heap into max heap, the array elements are:

89	81	76	22	14	9	54	11

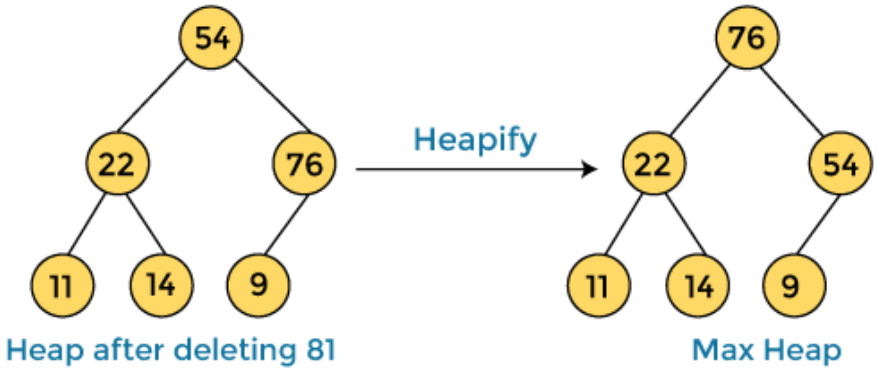
Next, we have to delete the root element **(89)** from the max heap. To delete this node, we have to swap it with the last node, i.e. **(11).** After deleting the root element, we again have to heapify it to convert it into max heap.



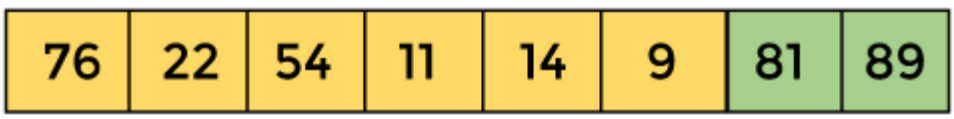
After swapping the array element **89** with **11,** and converting the heap into max-heap, the elements of array are -



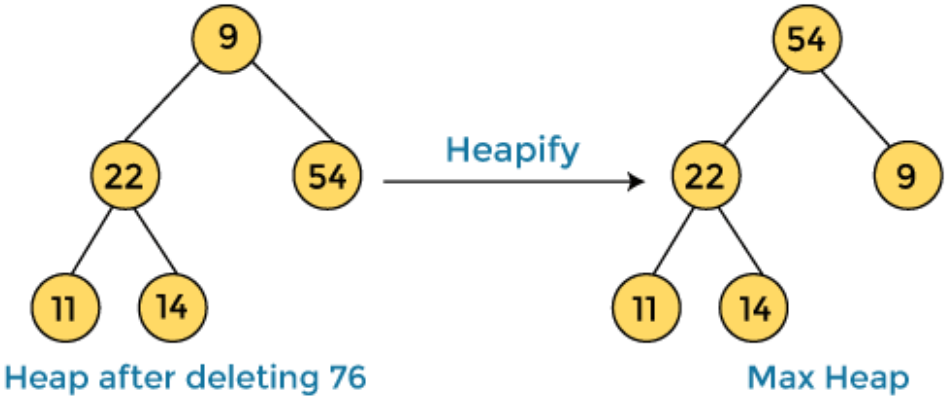
In the next step, again, we have to delete the root element **(81)** from the max heap. To delete this node, we have to swap it with the last node, i.e. **(54).** After deleting the root element, we again have to heapify it to convert it into max heap.



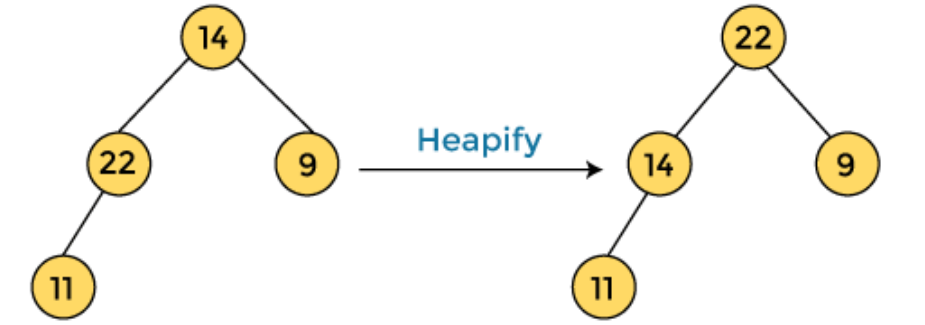
After swapping the array element **81** with **54** and converting the heap into max-heap, the elements of array are –



In the next step, we have to delete the root element **(76)** from the max heap again. To delete this node, we have to swap it with the last node, i.e. **(9).** After deleting the root element, we again have to heapify it to convert it into max heap.

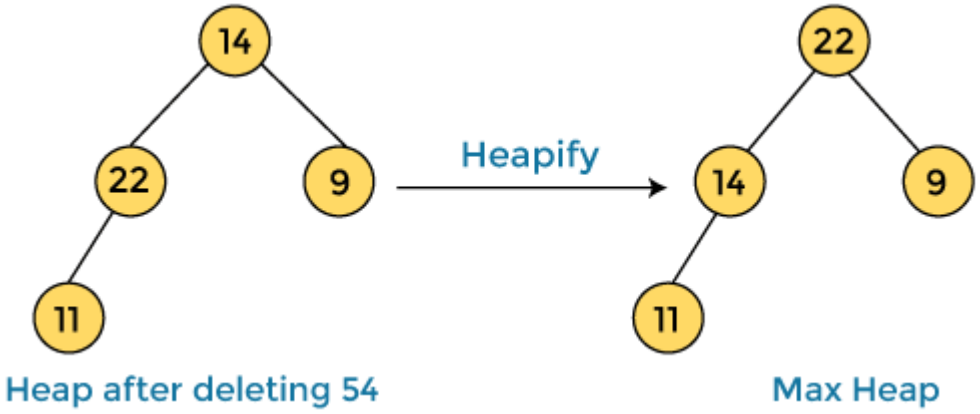


After swapping the array element **76** with **9** and converting the heap into max-heap, the elements of array are –

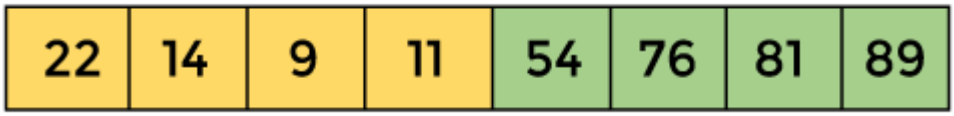


54	22	9	11	14	76	81	89

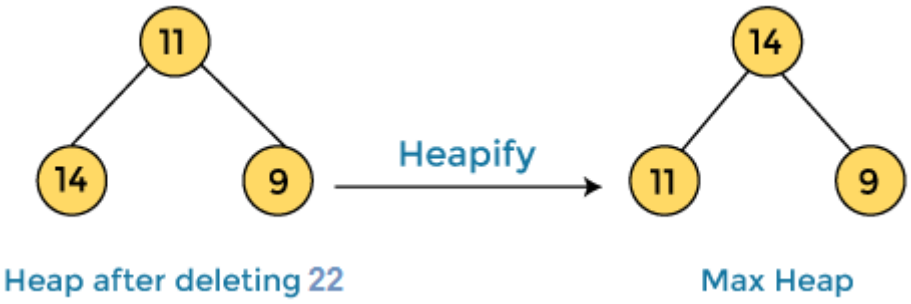
In the next step, again we have to delete the root element **(54)** from the max heap. To delete this node, we have to swap it with the last node, i.e. **(14).** After deleting the root element, we again have to heapify it to convert it into max heap.



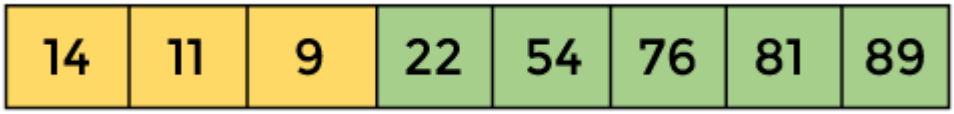
After swapping the array element **54** with **14** and converting the heap into max-heap, the elements of array are -



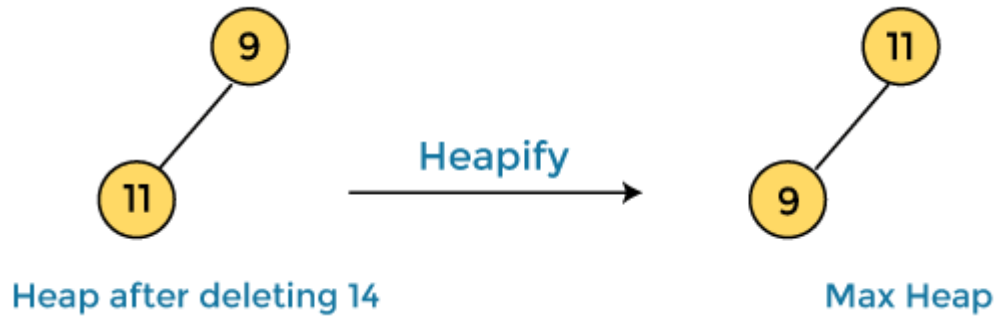
In the next step, again we have to delete the root element **(22)** from the max heap. To delete this node, we have to swap it with the last node, i.e. **(11).** After deleting the root element, we again have to heapify it to convert it into max heap.



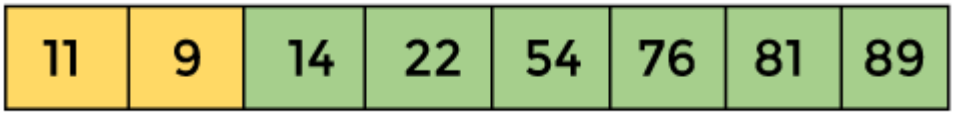
After swapping the array element **22** with **11** and converting the heap into max-heap, the elements of array are –



In the next step, again we have to delete the root element **(14)** from the max heap. To delete this node, we have to swap it with the last node, i.e. **(9).** After deleting the root element, we again have to heapify it to convert it into max heap.



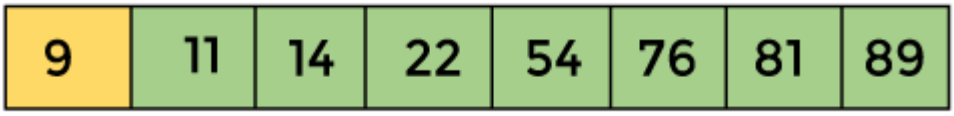
After swapping the array element **14** with **9** and converting the heap into max-heap, the elements of array are -



In the next step, again we have to delete the root element **(11)** from the max heap. To delete this node, we have to swap it with the last node, i.e. **(9).** After deleting the root element, we again have to heapify it to convert it into max heap.



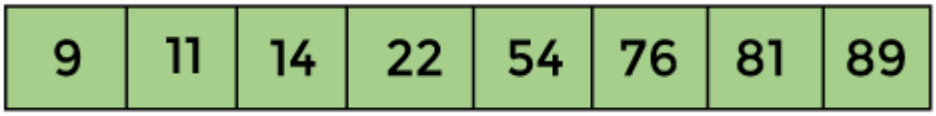
After swapping the array element **11** with **9,** the elements of array are -



Now, heap has only one element left. After deleting it, heap will be empty.



After completion of sorting, the array elements are -



Now, the array is completely sorted.

C Program

```
#include <stdio.h>
void swap(int* a, int* b)
{

      int temp = *a;
      *a = *b;
      *b = temp;
}
void heapify(int arr[], int N, int i)
{
      int largest = i;
      int left = 2 * i + 1;
      int right = 2 * i + 2;
      if (left < N && arr[left] > arr[largest])
            largest = left;
```

```
      if (right < N && arr[right] > arr[largest])
            largest = right;
      if (largest != i) {
            swap(&arr[i], &arr[largest]);
            heapify(arr, N, largest);
      }
}

void heapSort(int arr[], int N)
{
      for (int i = N / 2 - 1; i >= 0; i--)
            heapify(arr, N, i);
      for (int i = N - 1; i >= 0; i--) {
            swap(&arr[0], &arr[i]);
            heapify(arr, i, 0);
      }
}

void printArray(int arr[], int N)
{
      for (int i = 0; i < N; i++)
            printf("%d ", arr[i]);
      printf("\n");
}
int main()
{
      int arr[] = { 12, 11, 13, 5, 6, 7 };
      int N = sizeof(arr) / sizeof(arr[0]);

      // Function call
      heapSort(arr, N);
      printf("Sorted array is\n");
      printArray(arr, N);
}
```

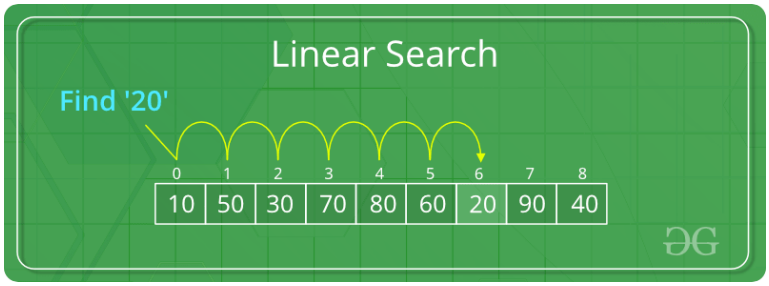
Time Complexity

- **Best Case Complexity -** It occurs when there is no sorting required, i.e. the array is already sorted. The best-case time complexity of heap sort is **O(n logn).**
- **Average Case Complexity -** It occurs when the array elements are in jumbled order that is not properly ascending and not properly descending. The average case time complexity of heap sort is **O(n log n).**
- **Worst Case Complexity -** It occurs when the array elements are required to be sorted in reverse order. That means suppose you have to sort the array elements in ascending

order, but its elements are in descending order. The worst-case time complexity of heap sort is **O(n log n).**

8. Linear Search

- Linear Search is defined as a sequential search algorithm that starts at one end and goes through each element of a list until the desired element is found, otherwise the search continues till the end of the data set. It is the easiest searching algorithm.



Algorithm

- Start from the leftmost element of arr[] and compare x with each element of arr[]
- If x matches with an element, return the index.
- If x doesn't match with any of the elements, return -1.

C Program

```
#include <stdio.h>
int search(int arr[], int N, int x)
{
      int i;
      for (i = 0; i < N; i++)
            if (arr[i] == x)
                  return i;
      return -1;
}
int main(void)
{
      int arr[] = { 2, 3, 4, 10, 40 };
      int x = 10;
      int N = sizeof(arr) / sizeof(arr[0]);
      int result = search(arr, N, x);
      (result == -1)
            ? printf("Element is not present in array")
            : printf("Element is present at index %d", result);
      return 0;
}
```

Output:

```
Element is present at index 3
```

Time complexity: O(N)

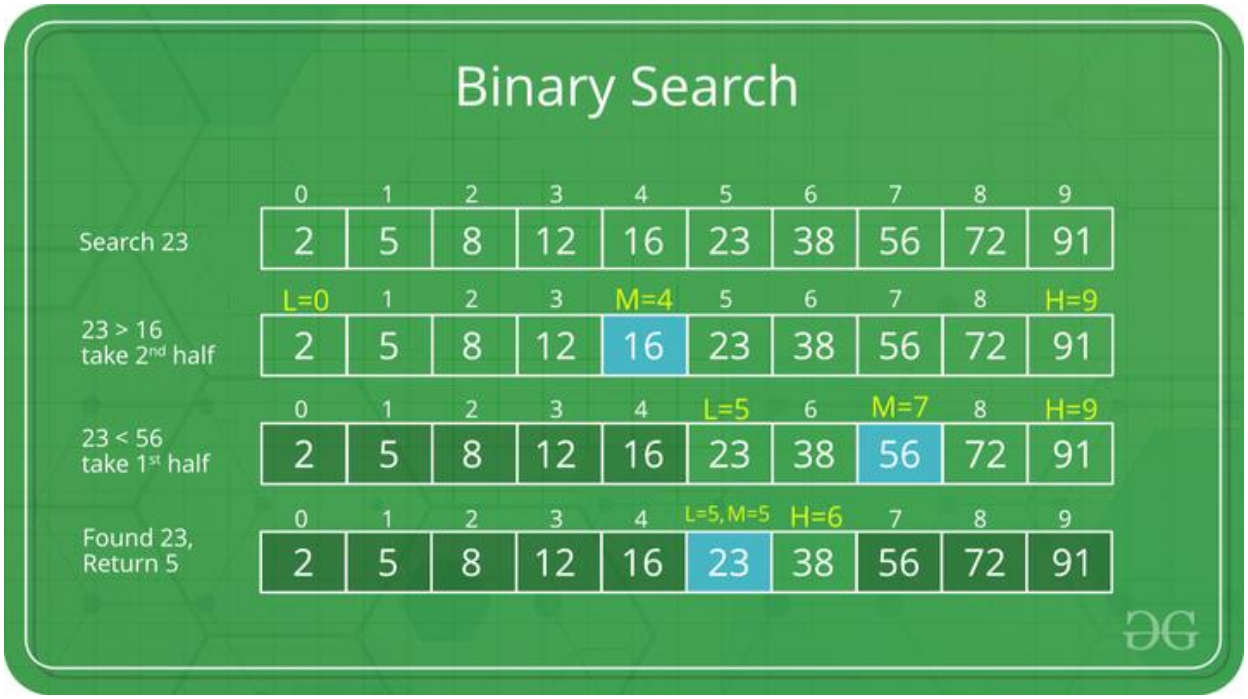
9. Binary Search:

- Binary Search is a searching algorithm used in a sorted array by repeatedly dividing the search interval in half of the array.
- The idea of binary search is to use the information that the array is sorted and reduce the time complexity to O(Log n).

Algorithm:

- Begin with the mid element of the whole array as a search key.
- If the value of the search key is equal to the item then return an index of the search key.
- Or if the value of the search key is less than the item in the middle of the interval, narrow the interval to the lower half.
- Otherwise, narrow it to the upper half.
- Repeatedly check from the second point until the value is found or the interval is empty.

Example:



C Program

```
#include <stdio.h>
int binarySearch(int arr[], int l, int r, int x)
{
	if (r >= l) {
		int mid = l + (r - l) / 2;
		if (arr[mid] == x)
			return mid;
		if (arr[mid] > x)
			return binarySearch(arr, l, mid - 1, x);
		return binarySearch(arr, mid + 1, r, x);
	}
	return -1;
}
int main(void)
{
	int arr[] = { 2, 3, 4, 10, 40 };
	int n = sizeof(arr) / sizeof(arr[0]);
	int x = 10;
	int result = binarySearch(arr, 0, n - 1, x);
	(result == -1)? printf("Element is not present in array")
		: printf("Element is present at index %d", result);
	return 0;
}
```

Output:

```
Element is present at index 3
```

Time Complexity: O(log n)

=000=